

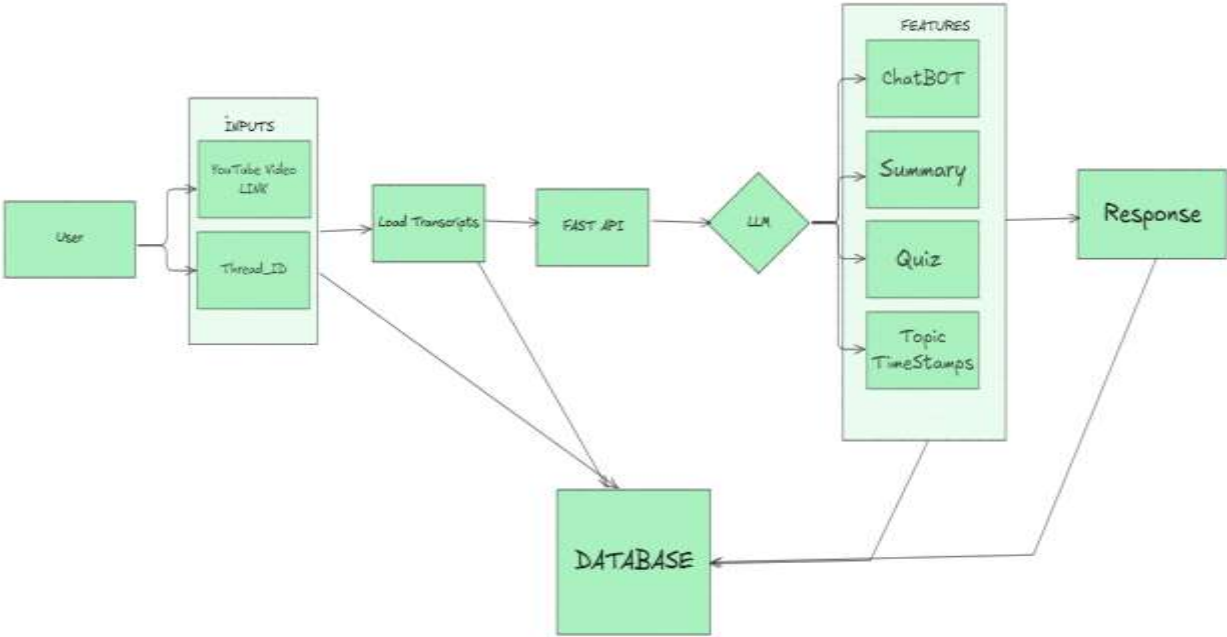
Department of Master of Computer Application

Session 2025-2026

Progress Report of Mini-Project work – 1

Name of the Projectees	1. Pranay S Gundu 2. Kushal K Chavan
Name of the Guide	Dr. Anup Bhangе
Department	Master of Computer Application
Date of submission	23-09-2025
Title	TubeTalk.ai
Report for the Period	13-08-2025 to 23-09-2025

Precise Report of the Project work done during the above period (Brief)	
1	Study the various paper related to research area 1.Scope & trends — Recent Blogs focus on combining Retrieval-Augmented Generation (RAG) with structured schemas to handle any written Content 2. Transcription + retrieval — Best results come from accurate transcripts followed by chunked retrieval (semantic search) rather than sending full transcripts to the LLM. 3.Privacy & cost — Research highlights stored transcripts and using caching/checkpointing to reduce API tokens and latency.
2	Find out the various Techniques / Algorithm 1. RAG (Retrieval-Augmented Generation) — Store transcript chunks in a vector DB; retrieve relevant chunks per query and feed to LLM for focused answers. 2. Structured Output / Schema Enforcement — Ask LLM to output JSON with fields (summary, key_points[], timestamps[], quiz[]) for predictable parsing and DB storage. 3. Caching — Cache embeddings, retrieval results, and common LLM outputs to avoid repeated token usage for the same video/thread. 4. Checkpointer / Conversation History Checkpointing — Periodically checkpoint user-LLM sessions (store compressed context or summary) so long chats don't resend entire history. ○
3	Analysis of existing algorithms and its weakness Non-RAG (naïve LLM on whole transcript) uses many tokens — Cost and latency explode for long videos; prompts can exceed model limits. No caching → repeated cost — Systems that regenerate summaries/answers each time increase API bills and slow UX

4	Analysis of existing algorithms with our system 1. RAG reduces token usage — TubeTalk uses chunked retrieval so only relevant chunks go to LLM, lowering cost and improving focus. 2. Structured schema yields reliable features — By enforcing JSON schema (summary, quiz, timestamps), TubeTalk.ai stores and serves outputs cleanly to the UI and DB. 3. Caching & checkpointing improve speed & cost — Cached embeddings and checkpoints let users re-open sessions or regenerate quizzes without new full LLM runs. 4. Timestamp extraction + overlap reduces boundary errors — Our chunk overlap and timestamp alignment gives users accurate jump-to moments. 5. Fallback & logging increase robustness — If LLM fails, TubeTalk falls back to stored cached outputs or lightweight extractive summaries and logs failures for debug and retry.
5	Publish the Introductory/ Survey paper on research topic: Yes/No
6	Explore the flow of Project – TubeTalk.ai  <pre> graph LR User[User] --> Inputs[INPUTS YouTube Video LINK Thread_ID] Inputs --> LoadTranscripts[Load Transcripts] LoadTranscripts --> FASTAPI[FAST API] FASTAPI --> LLM{LLM} LLM --> Features[FEATURES ChatBOT Summary Quiz Topic TimeStamps] Features --> Response[Response] Inputs --> Database[DATABASE] LoadTranscripts --> Database Features --> Database </pre> Flow of TubeTalk.ai 1. User Input & Authentication ○ User provides YouTube video link and Thread_ID. ○ Request goes through frontend → backend (FastAPI). ○ Session is authenticated using JWT-based tokens for secure access. 2. Transcript Loading ○ System fetches and loads YouTube transcripts via backend service. ○ Transcripts are stored temporarily and/or cached in the database for reuse.

	3. Processing via LLM ○ FastAPI sends transcripts to the LLM. ○ LLM generates multiple feature outputs: ▪ Chatbot (Q&A on video content) ▪ Summary (shortened version of content) ▪ Quiz (auto-generated questions) ▪ Topic Timestamps (segmented highlights) 4. Database Storage ○ All outputs (transcripts, summaries, quizzes, timestamps) are encrypted and stored in the database. ○ Each entry linked with Thread_ID for retrieval. 5. Response Delivery ○ Processed results (chatbot answers, summaries, quizzes, timestamps) are returned to the user as a structured response. ○ Database ensures fast reload for repeated queries. 6. Error Handling & Logging ○ Any failed transcript loads or API errors are logged. ○ If LLM fails, fallback response is triggered with stored data from DB.
7	Research Papers Published in the refereed/national/international journals in this duration (please attach copy): Yes/No Yes
8	Research Papers Published in Conference Proceedings/ Seminars in this duration (please attach copy) Yes/No Yes
9	Conferences/Seminars/workshops attended in this duration: Yes/No Yes
10	Any other achievements No

Signature of students

Signature of Guide

Signature of Project In charge